

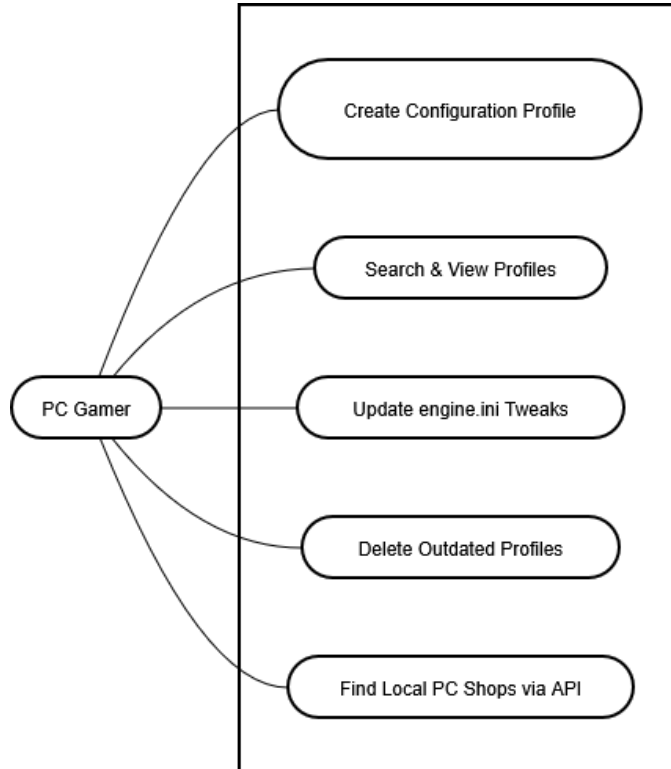
Project Purpose and Audience

- **Purpose:** Engine Optimizer is a local desktop performance optimization manager designed to manage and share PC game optimization profiles (such as Unreal Engine engine.ini tweaks) to address performance issues.
- **Audience:** The primary target audience includes PC gaming enthusiasts, hardware overclockers, and developers looking for a structured way to manage environmental variables across different hardware setups.

Requirements:

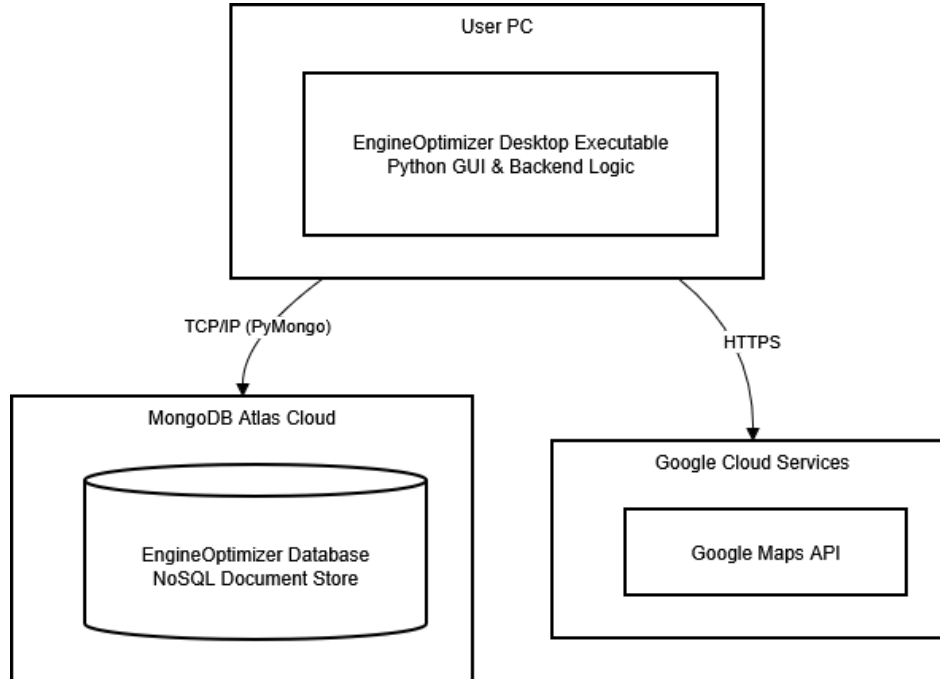
ID	Type	Requirement Description
REQ-1	Functional	The system shall allow users to upload, store, and edit flexible, text-heavy engine configuration data as NoSQL documents.
REQ-2	Functional	The system shall retrieve saved optimization documents based on the specific game title and hardware configuration.
REQ-3	Functional	The system shall allow users to permanently delete their saved configuration documents from the cloud database.
REQ-4	Non-Functional (Performance)	The system must retrieve a saved profile from the database and render it in the UI in under 2 seconds.
REQ-5	Non-Functional (Scalability)	The database architecture must utilize horizontal scaling to support concurrent read/write operations for multiple users without performance degradation.

Scenario Viewpoint:



This Use Case Diagram illustrates the core functionalities of the OptiGraph desktop application from the perspective of the primary actor: the PC Gamer. The user interacts with the GUI to perform CRUD operations on their optimization profiles (creating, viewing, updating, and deleting settings for specific games and hardware). Additionally, the user can trigger the store finder use case, which utilizes the Google Maps API to locate nearby PC hardware shops.

Physical Viewpoint:



This Deployment Diagram details the physical layout of the OptiGraph architecture. The system consists of a User PC running the compiled Python desktop executable (which contains both the GUI and the backend controllers). This local application acts as a direct client to the MongoDB Atlas Cloud infrastructure, communicating via TCP/IP using the pymongo driver to store and retrieve the engine.ini configurations as JSON-like documents. Finally, the application makes direct HTTPS requests to the external Google Maps API for the store finder feature.